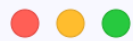




Classificador Bayesiano Multivariado
Introdução ao Reconhecimento de Padrões
Turma TB

Gabriel Vaz Cançado Ferreira
Matrícula: 2021015194

1. Carregar os dados.



```
df = pd.read_csv('heart.dat', sep = ' ', header = None)
```

2. Separar os dados em 90% para treinamento e 10% para teste.



```
train = df.loc[0:int(0.9*len(df)),]  
test = df.loc[int(0.9*len(df))+1:,]  
  
X_train = pd.DataFrame()  
X_test = pd.DataFrame()  
for i in range(13):  
    X_train[i] = train[i]  
    X_test[i] = test[i]  
  
y_train = pd.DataFrame(data=train[13])  
y_test = pd.DataFrame(data=test[13])
```

3. Estimar a densidade multivariada para as duas classes utilizando a expressão da figura 0.1 vista em sala de aula.

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n |\Sigma|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_2 & \cdots & \rho_{1n}\sigma_1\sigma_n \\ \rho_{21}\sigma_2\sigma_1 & \sigma_2^2 & \cdots & \rho_{2n}\sigma_2\sigma_n \\ \vdots & \vdots & \cdots & \vdots \\ \rho_{n1}\sigma_n\sigma_1 & \rho_{n2}\sigma_n\sigma_2 & \cdots & \sigma_n^2 \end{bmatrix}$$

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix}$$

Figura 0.1: PDF multivariada

Implementação da função densidade de probabilidade:



```
def pdf(x, mu, sigma, n):
    const = 1 / np.sqrt(((2*np.pi)**n) * np.linalg.det(sigma))
    diff = (x - mu).reshape(-1, 1)
    expo = -0.5 * diff.T @ np.linalg.inv(sigma) @ diff
    return float(const * np.exp(expo))
```

Cálculo dos parâmetros a serem utilizados na pdf:



```
y_train = y_train.squeeze()
classes = y_train.unique()
n = X_train.shape[1]

priors = {c: (y_train == c).mean() for c in classes}

params = {}
for c in classes:
    X_c = X_train[y_train == c]
    mu_c = X_c.mean().values
    sigma_c = np.cov(X_c.T)
    params[c] = (mu_c, sigma_c)
```

4. **Aplicar o classificador ao conjunto de teste e calcular a acurácia. A acurácia é o número de classificações corretas do conjunto de testes dividido pelo número total de amostras do mesmo conjunto.**

Para cada linha do conjunto de teste, aplicamos a função definida anteriormente, calculando a probabilidade daquele vetor pertencer a cada uma das classes, de acordo com a densidade multivariada estimada. Em seguida, multiplicamos essa verossimilhança pela probabilidade a priori da classe, obtendo a probabilidade a posteriori. A decisão final é feita atribuindo ao vetor a classe que apresentar a maior probabilidade a posteriori, conforme estabelece a regra de Bayes.



```
y_pred = []

for i, row in X_test.iterrows():
    x = row.values
    posteriors = {}
    for c in classes:
        mu, sigma = params[c]
        likelihood = pdf(x, mu, sigma, n)
        posteriors[c] = likelihood * priors[c]

    y_pred.append(max(posteriors, key=posteriors.get))

y_pred = pd.Series(y_pred, index=X_test.index)
```

Por fim, calculamos a acurácia:



```
from sklearn.metrics import accuracy_score

print("Acurácia:", accuracy_score(y_test, y_pred))
```

Acurácia: 0.7692307692307693

5. **Variar o tamanho da amostra de treinamento para 70% e 20% e repetir o experimento.**
 - a. 70%
Acurácia: 0.8375
 - b. 20%
Acurácia: 0.7674418604651163
6. **Discorra brevemente sobre o efeito do número de amostras de treinamento no resultado da classificação.**

Notamos que houve um aumento significativo na acurácia ao reduzirmos o tamanho conjunto de treinamento para 70% e uma redução para bem próximo do valor inicial conforme reduziu-se o conjunto de treino para 20%, demonstrando que a performance do classificador de bayes multivariado tem melhor performance em valores intermediários de tamanho do conjunto de treinamento.